GUÍA DE CLASE · JAVA

Pruebas unitarias en Java con JUnit 5

Del “funciona en mi máquina” al código que se demuestra: pruebas, cobertura con JaCoCo e IA como copiloto.

```
@Test
void estaAprobado_notaExactamenteTres_retornaTrue() {
    assertTrue(calculadora.estaAprobado(3.0));
}
✗ expected: <true> but was: <false>  →  ✓ bug encontrado
```

Prof. Edwin Rozo Gómez
Ingeniero de software · Instructor

2 horas
40 min teoría · 80 min práctica

Nivel
Principiante (POO básica)

ANTES DE EMPEZAR

Mapa de la sesión

Esta guía cuenta **una sola historia** de principio a fin. Cada sección responde una pregunta y deja lista la siguiente: por qué probar → qué es una prueba → con qué herramienta → cómo se hace y se mide → cómo acelerar con IA → hazlo tú → entrégalo en GitHub.

EL HILO CONDUCTOR: EL APRENDIZ QUE SACÓ 3.0

Un instituto usa una pequeña clase Java, `CalculadoraNotas`, para calcular la nota final de sus aprendices (30 % seguimiento, 30 % parcial, 40 % proyecto) y decidir si aprueban. La regla es clara: **aprueba con 3.0 o más**. Un aprendiz obtiene exactamente **3.0**... y el sistema lo reporta como **reprobado**. Nadie lo notó porque "el programa compilaba y funcionaba". En esta sesión vamos a **escribir las pruebas que habrían atrapado ese error**, medir qué tanto del código revisan y usar IA sin dejar que nos engañe. En el taller harás lo mismo con un **carrito de compras** que también esconde un bug, y lo entregarás en **tu propio repositorio de GitHub**.

Tiempo	Bloque	Tipo	Pregunta que responde
Antes de clase	0. Preparación: cuenta de GitHub, Git e IntelliJ conectados (≈ 25 min, en casa o en el aula)	Práctica	¿Tengo todo listo para trabajar?
0:00 – 0:10	Apertura: el caso del aprendiz con 3.0	Teoría	¿Qué problema vamos a resolver?
0:10 – 0:22	1. Importancia de las pruebas y métricas reales	Teoría	¿Por qué vale la pena probar?
0:22 – 0:35	2. Qué son las pruebas unitarias · 3. JUnit 5 + JaCoCo	Teoría	¿Qué es exactamente y con qué lo hacemos?
0:35 – 1:15	4. Paso a paso guiado: abrir y publicar el proyecto, probar `CalculadoraNotas` y medir cobertura	Práctica	¿Cómo se escribe, se ejecuta y se mide?
1:15 – 1:23	5. La IA en el proceso (demo en vivo)	Práctica	¿Cómo acelero sin perder el control?
1:23 – 1:53	6. Ejercicio-taller individual: `CarritoCompras`	Práctica	¿Puedo hacerlo solo?
1:53 – 2:00	7. Entrega en GitHub y cierre	Práctica	¿Cómo demuestro lo que hice?

Distribución exacta en clase: 35 minutos de teoría (0 h 35 min) y 85 minutos de práctica (1 h 25 min). La Sección 0 se hace antes de la clase.

Modalidad	Detalle
Paso a paso (Sección 4)	**Individual**, cada uno en su computador, siguiendo al instructor en pantalla.
Taller (Secciones 6 y 7)	**Individual.** Puedes preguntarle a un compañero, pero cada uno escribe su propio código y sube su propio repositorio.
Qué entregas	**Un solo enlace**: el de tu repositorio público en GitHub (lo explica la Sección 7).

Requisitos previos

Necesitas	Detalle
Java JDK 17 o superior	Verifica con `java -version` en la terminal.
IntelliJ IDEA (Community sirve)	Trae Maven, JUnit, un medidor de cobertura y la conexión con GitHub integrados.
Un correo electrónico al que tengas acceso	Para crear tu cuenta de GitHub (Sección 0).
Conocimientos	Clases, métodos, `if`, excepciones básicas (`throw new …`). No necesitas saber nada de Git ni de GitHub.
Material de apoyo	Archivo `notas-app-estudiante.zip` entregado por el instructor.

SECCIÓN 0 Antes de clase · ≈ 25 min

Preparación: GitHub desde cero

Al final de la clase vas a entregar tu trabajo como un **repositorio de GitHub**. Si nunca lo has usado, aquí empiezas. Haz esta sección **antes** de la clase: crear la cuenta requiere confirmar tu correo y no queremos perder tiempo de práctica en eso.

ANALOGÍA: GIT ES EL HISTORIAL, GITHUB ES LA NUBE

Git es un programa en tu computador que guarda "fotos" de tu proyecto cada vez que tú se lo pides (cada foto se llama **commit**). **GitHub** es una página web donde subes esas fotos para que otros las vean (subir se llama **push**). Es como guardar versiones de un documento y luego compartir el enlace de la carpeta en la nube: tu instructor abrirá tu enlace y verá tu código, tus pruebas y tus capturas.

0.1 Crear tu cuenta de GitHub (8 min)

1. Entra a **github.com** y haz clic en **Sign up**.
2. Escribe tu correo, una contraseña y un **nombre de usuario**. Usa algo profesional, sin espacios, por ejemplo `laura-gomez-dev`. Ese nombre aparecerá en el enlace que vas a entregar.
3. Resuelve la verificación ("no soy un robot") y escribe el **código que llega a tu correo**.
4. Si te pregunta por un plan, elige el gratuito (**Free**). Si te pide activar la verificación en dos pasos, puedes hacerlo con la app **GitHub Mobile** o con una app de autenticación.

0.2 Instalar Git (7 min)

1. Entra a **git-scm.com** › **Download for Windows** y descarga el instalador de 64 bits.
2. Ejecútalo y presiona **Next** en todas las pantallas (las opciones por defecto están bien) y al final **Finish**.
3. Si IntelliJ estaba abierto, **ciérralo y vuelve a abrirlo** para que reconozca Git.
4. Comprueba: en IntelliJ abre la terminal (**View › Tool Windows › Terminal**, o `Alt + F12`) y escribe:

```
git --version
git version 2.xx.x.windows.x      ← cualquier número de versión significa que quedó instalado
```

Alternativa: si no puedes instalar programas (computador del aula), en IntelliJ ve a **File › Settings › Version Control › Git**; si dice que Git no está instalado, aparece un enlace **Download and Install**.

0.3 Decirle a Git quién eres (2 min)

Cada commit lleva tu nombre. En la misma terminal escribe (con tus datos y entre comillas):

```
git config --global user.name "Laura Gómez"
git config --global user.email "el-correo-con-el-que-creaste-github@ejemplo.com"
```

Usa el **mismo correo de tu cuenta de GitHub**, así tus commits aparecen asociados a tu perfil. No muestra ningún mensaje si todo salió bien.

0.4 Conectar IntelliJ con tu cuenta de GitHub (5 min)

1. En IntelliJ: **File › Settings** (`Ctrl + Alt + S`) › **Version Control › GitHub**.
2. Haz clic en **+** › **Log In via GitHub…** Se abre el navegador.
3. Inicia sesión en GitHub si te lo pide y haz clic en **Authorize JetBrains** (o **Authorize in GitHub**).
4. Vuelve a IntelliJ: tu usuario debe aparecer en la lista. Clic en **OK**.

Lista de verificación de la Sección 0

✔	Antes de llegar a clase…
☐	Tengo usuario y contraseña de GitHub y puedo iniciar sesión en github.com.
☐	`git --version` muestra una versión en la terminal de IntelliJ.
☐	Configuré `user.name` y `user.email`.
☐	En **Settings › Version Control › GitHub** aparece mi cuenta.

> **¿ALGO NO FUNCIONÓ?**
> No te preocupes: avísale al instructor **al inicio de la clase**, no al final. Si la conexión con IntelliJ falla, existe un plan B que solo usa el navegador (Sección 7, “Si algo falla”).

SECCIÓN 1 0:10 – 0:22 · Teoría

¿Por qué probar? Lo que dicen los datos

Probar no es un trámite: es la forma más barata de encontrar errores. Y el momento en que los encuentras lo cambia todo.

US$ 2,41 billones
costo anual estimado de la mala calidad del software en EE. UU. (2022).
CISQ, 2022

US$ 59.500 M
costo anual de una infraestructura de pruebas deficiente en EE. UU.; ~US$ 22.200 M evitables mejorando las pruebas.
NIST, 2002

40 % – 90 %
menos defectos en equipos de Microsoft e IBM que escribieron pruebas primero (TDD), a cambio de 15 – 35 % más tiempo inicial.
Nagappan et al., 2008

≈ 100×
más caro corregir un defecto después de entregado que en requisitos/diseño (proyectos grandes).
Boehm & Basili, 2001

LEAMOS LOS NÚMEROS CON CRITERIO

Las cifras de "cuántas veces más caro" varían según el proyecto y el estudio; en proyectos pequeños la diferencia es menor. Lo que sí se repite en toda la literatura es la **tendencia**: cuanto más tarde aparece un error, más cuesta. Un ingeniero cita la fuente y el contexto, no solo el número.

Del caso del aprendiz a la curva de costo

¿Cuándo se detecta el bug del 3.0?	¿Qué pasa?	Costo relativo
Al escribir una prueba unitaria	La prueba falla en segundos. Cambias `>` por `>=`.	Minutos del desarrollador.
En pruebas de QA / integración	Alguien reporta, otro reproduce, se asigna, se corrige y se vuelve a probar.	Horas de varias personas.
En producción	El aprendiz reclama, coordinación académica revisa notas de todo el grupo, se corrigen certificados.	Días, reputación y confianza.

La pirámide de pruebas

No todas las pruebas son iguales. La recomendación clásica (Google Testing Blog, 2015) es una proporción aproximada de **70 % unitarias, 20 % de integración y 10 % de extremo a extremo**: las unitarias son rápidas, baratas y señalan exactamente dónde está el error.



¿Cuánta cobertura es “buena”?

La **cobertura** mide qué porcentaje del código ejecutan tus pruebas (la medimos en la Sección 4). Google publicó en 2020 una guía de referencia interna:

Cobertura	Valoración en la guía de Google
60 %	Aceptable
75 %	Recomendable (“commendable”)
90 %	Ejemplar (“exemplary”)

CONEXIÓN CON LO QUE SIGUE

Ya sabemos **por qué** conviene encontrar el bug del 3.0 lo antes posible. La herramienta para hacerlo en minutos es la **prueba unitaria**. ¿Qué es exactamente?

SECCIÓN 2 0:22 – 0:29 · Teoría

¿Qué son y cómo se usan las pruebas unitarias?

Una **prueba unitaria** es un pequeño programa que ejecuta **una unidad** de tu código (normalmente un método) con datos conocidos y **verifica automáticamente** que el resultado sea el esperado.

ANALOGÍA: LA FÁBRICA DE CARROS

Antes de ensamblar un carro, cada pieza se prueba por separado: los frenos se prueban en un banco, el motor en otro. Si el carro ensamblado falla, nadie sabe qué pieza fue. Si cada pieza tiene su prueba, el fallo señala la pieza exacta. En nuestro código, `estaAprobado()` es una pieza; su prueba unitaria es su banco de pruebas.

Anatomía: el patrón AAA (Arrange – Act – Assert)

```java
@Test
void calcularNotaFinal_notasValidas_retornaPromedioPonderado() {
    // Arrange (Preparar): los datos de entrada y el objeto a probar
    CalculadoraNotas calculadora = new CalculadoraNotas();

    // Act (Actuar): ejecutar UNA acción
    double resultado = calculadora.calcularNotaFinal(4.0, 3.0, 5.0);

    // Assert (Afirmar): comparar lo obtenido con lo esperado
    assertEquals(4.1, resultado, 0.001);   // 4.0·0.3 + 3.0·0.3 + 5.0·0.4 = 4.1
}
```

Las preguntas que una buena prueba responde

Tipo de caso	Pregunta	Ejemplo con `estaAprobado()`
Camino feliz	¿Funciona con datos normales?	3.5 → aprueba
Caso negativo	¿Rechaza lo que debe rechazar?	2.9 → reprueba
Caso límite	¿Qué pasa justo en la frontera?	**3.0 → aprueba** (aquí vive el bug)
Entrada inválida	¿Se defiende de datos imposibles?	5.1 o −0.1 → excepción

Principios FIRST

Principio	Significado
Fast (rápida)	Milisegundos. Si tarda, nadie la ejecuta.
Independent	No depende de otra prueba ni del orden. Por eso usamos `@BeforeEach` para crear un objeto nuevo cada vez.
Repeatable	Mismo resultado siempre: sin internet, sin hora del sistema, sin base de datos.
Self-validating	Dice sola si pasó o falló (verde/rojo). Nada de mirar un `println`.
Timely (oportuna)	Se escribe junto con el código, no meses después.

Unitaria vs. otras pruebas

Tipo	Qué prueba	Ejemplo en nuestro caso	Velocidad
Unitaria	Un método aislado	`estaAprobado(3.0)`	Milisegundos
Integración	Varias piezas juntas	Calculadora + base de datos de notas	Segundos
Extremo a extremo	El sistema completo como usuario	Docente ingresa notas en la web y ve el reporte	Minutos

CONVENCIÓN DE NOMBRES QUE USAREMOS

`metodo_escenario_resultadoEsperado` → `estaAprobado_notaExactamenteTres_retornaTrue`.
Cuando una prueba falle, su nombre ya te dice qué se rompió.

SECCIÓN 3 0:29 – 0:35 · Teoría

¿Qué framework vamos a usar?

Tres piezas que trabajan juntas: **JUnit 5** escribe y ejecuta las pruebas, **JaCoCo** mide la cobertura y **Maven** descarga todo y lo conecta. IntelliJ IDEA lo integra con botones.

Herramienta	Rol	Por qué esta
JUnit 5 (Jupiter)	Escribir y ejecutar pruebas	Estándar de la industria en Java; integrado en IntelliJ, Maven, Gradle y Spring Boot. Su sucesor, JUnit 6 (2025), conserva la misma API Jupiter: todo lo de esta guía funciona igual.
JaCoCo	Medir cobertura	Gratuito, el más usado en Java, genera reportes HTML con colores línea por línea.
Maven	Gestionar dependencias y ejecutar	Un archivo (`pom.xml`) describe el proyecto; `mvn test` ejecuta todo igual en cualquier computador.
IntelliJ IDEA	Entorno de trabajo	Botón ▶ para correr pruebas y “Run with Coverage” para ver cobertura sin configurar nada.

Anotaciones de JUnit 5 que usaremos hoy

Anotación	Para qué sirve
`@Test`	Marca un método como prueba.
`@BeforeEach`	Se ejecuta antes de **cada** prueba (crea un objeto limpio → principio *Independent*).
`@DisplayName("…")`	Nombre legible en español que aparece en el reporte.
`@ParameterizedTest` + `@CsvSource`	Una prueba que se ejecuta con muchos datos (ideal para escalas y límites).
`@ValueSource`	Lista simple de valores para una prueba parametrizada.

Aserciones (los “Assert” del patrón AAA)

Aserción	Verifica que...
`assertEquals(esperado, real)`	dos valores son iguales.
`assertEquals(4.1, real, 0.001)`	dos `double` son iguales con un margen (los decimales no son exactos en binario).
`assertTrue(cond)` / `assertFalse(cond)`	una condición es verdadera / falsa.
`assertThrows(Excepcion.class, () -> …)`	el código lanza la excepción esperada.

¿Qué es la cobertura y qué mide JaCoCo?

Métrica	Pregunta	Color en el reporte
Líneas	¿Qué líneas se ejecutaron al menos una vez?	verde todo cubierto · amarillo rama parcial · rojo nunca se ejecutó
Ramas (branches)	En cada `if`, ¿se probó el camino verdadero **y** el falso?	
Métodos	¿Qué métodos se llamaron al menos una vez?	—

CONEXIÓN CON LO QUE SIGUE

Ya tenemos el qué (pruebas unitarias con AAA), el con qué (JUnit 5 + JaCoCo) y el para qué (atrapar el bug del 3.0 a tiempo). Ahora lo hacemos, paso a paso.

SECCIÓN 4 0:35 – 1:15 · Práctica guiada

Paso a paso: probando `CalculadoraNotas`

Primero dejamos el proyecto abierto y publicado en tu GitHub. Luego construimos las pruebas en el orden en que un desarrollador real lo haría: la prueba más simple, medir, cerrar huecos... y en el camino aparece el bug.

---

1 Abrir el proyecto del zip (3 min)

1. Busca `notas-app-estudiante.zip` en tu carpeta de Descargas › clic derecho › **Extraer todo...** › **Extraer**. (No trabajes con el zip sin extraer.)
2. En IntelliJ: **File › Open...** y selecciona la carpeta `notas-app-estudiante` (la que contiene el archivo `pom.xml`) › **OK**.
3. Si pregunta, elige **Trust Project**. Espera a que termine la barra de progreso de abajo: Maven está descargando JUnit y JaCoCo (necesita internet).
4. Si aparece un aviso amarillo *"Project JDK is not defined"*, haz clic en **Setup SDK** y elige un JDK 17 o superior.

Abre el panel **Project** (a la izquierda). Debes ver esta estructura; las dos clases ya están escritas, **no las crees de nuevo**:

```
notas-app-estudiante/
├── .gitignore        ← le dice a Git qué NO subir (carpetas target/ y .idea/)
├── README.md         ← la portada de tu repositorio (la completas en la Sección 7)
├── pom.xml           ← configuración de Maven: JUnit 5 + JaCoCo
└── src/
    ├── main/java/co/edu/notas/   ← código de producción: CalculadoraNotas.java,
CarritoCompras.java
    └── test/java/co/edu/notas/   ← aquí van las pruebas (por ahora vacía), mismo paquete
```

2 Publicar el proyecto en tu GitHub (7 min)

Lo publicamos **al inicio** para que, si algo falla con GitHub, haya tiempo de resolverlo antes de la entrega.

1. En el menú superior: **VCS › Share Project on GitHub**. (En la interfaz nueva, el menú está detrás del ícono ☰; si ves **Git** en lugar de **VCS**, usa **Git › GitHub › Share Project on GitHub**.)
2. En la ventana que aparece:
   - **Repository name:** `pruebas-unitarias-java`
   - **Private: sin marcar** (el repositorio debe ser **público** para que el instructor lo vea).
   - **Remote:** `origin` (déjalo así) › clic en **Share**.
3. Aparece **Add Files For Initial Commit**: verifica que estén marcados `.gitignore`, `README.md`, `pom.xml` y `src`, y clic en **Add**.
4. Abajo a la derecha aparece *"Successfully shared project on GitHub"* con un enlace. Ábrelo: ya tienes tu repositorio en internet.

COMPRUEBA

Tu enlace tiene la forma `https://github.com/tu-usuario/pruebas-unitarias-java` y en la página ves las carpetas del proyecto. Ese es el enlace que entregarás al final.

3 Entender el `pom.xml`: JUnit 5 + JaCoCo (2 min)

Ya viene configurado; no lo modifiques. Esto es lo que contiene:

pom.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>co.edu.notas</groupId>
  <artifactId>notas-app</artifactId>
  <version>1.0</version>

  <properties>
    <maven.compiler.release>17</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>5.11.4</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.2.5</version>
      </plugin>
      <plugin>
        <groupId>org.jacoco</groupId>
        <artifactId>jacoco-maven-plugin</artifactId>
        <version>0.8.12</version>
        <executions>
          <execution>
            <id>preparar-agente</id>
            <goals><goal>prepare-agent</goal></goals>
          </execution>
          <execution>
            <id>reporte</id>
            <phase>test</phase>
            <goals><goal>report</goal></goals>
```

```
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>
</project>
```

Qué hace cada parte: `junit-jupiter` trae JUnit 5 (solo para pruebas: `scope test`). `maven-surefire-plugin` ejecuta las pruebas. `jacoco-maven-plugin` se "engancha" a esas pruebas (`prepare-agent`) y al final genera el reporte de cobertura (`report`).

4 Conocer el código a probar (2 min)

Abre `src/main/java/co/edu/notas/CalculadoraNotas.java` y léelo con el instructor. **No lo modifiques todavía**: el bug del aprendiz con 3.0 está escondido aquí y lo vamos a descubrir con pruebas, no a ojo.

CalculadoraNotas.java

```java
package co.edu.notas;

public class CalculadoraNotas {

    public static final double NOTA_MINIMA = 0.0;
    public static final double NOTA_MAXIMA = 5.0;
    public static final double NOTA_APROBATORIA = 3.0;

    /** Calcula la nota final: 30% seguimiento + 30% parcial + 40% proyecto. */
    public double calcularNotaFinal(double seguimiento, double parcial, double proyecto) {
        validarNota(seguimiento);
        validarNota(parcial);
        validarNota(proyecto);
        double resultado = seguimiento * 0.30 + parcial * 0.30 + proyecto * 0.40;
        return Math.round(resultado * 10) / 10.0; // redondea a una décima
    }

    /** Aprueba quien obtiene 3.0 o más. */
    public boolean estaAprobado(double notaFinal) {
        validarNota(notaFinal);
        return notaFinal > NOTA_APROBATORIA;
    }

    /** Escala de desempeño: Superior, Alto, Básico, Bajo. */
    public String desempeno(double notaFinal) {
        validarNota(notaFinal);
        if (notaFinal >= 4.6) return "Superior";
        if (notaFinal >= 4.0) return "Alto";
        if (notaFinal >= 3.0) return "Básico";
        return "Bajo";
    }

    private void validarNota(double nota) {
        if (nota < NOTA_MINIMA || nota > NOTA_MAXIMA) {
            throw new IllegalArgumentException("La nota debe estar entre 0.0 y 5.0: " + nota);
        }
```

```
    }
}
```

La escala Superior / Alto / Básico / Bajo es la de valoración de desempeño usada en Colombia, con sus equivalencias en escala 0 – 5.

5 Primera prueba: el camino feliz (4 min)

Con `CalculadoraNotas.java` abierto, pon el cursor sobre el nombre de la clase › **Ctrl + Shift + T** › **Create New Test...** › Testing library: **JUnit 5** › **OK**. IntelliJ crea `CalculadoraNotasTest.java` en la carpeta `test`. **Borra todo su contenido** y pega esto:

```java
package co.edu.notas;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import org.junit.jupiter.params.provider.ValueSource;

import static org.junit.jupiter.api.Assertions.*;

class CalculadoraNotasTest {

    private CalculadoraNotas calculadora;

    @BeforeEach
    void setUp() {
        calculadora = new CalculadoraNotas();   // objeto nuevo para cada prueba
    }

    @Test
    @DisplayName("Calcula la nota final ponderada 30/30/40")
    void calcularNotaFinal_notasValidas_retornaPromedioPonderado() {
        // Arrange
        double seguimiento = 4.0, parcial = 3.0, proyecto = 5.0;
        // Act
        double resultado = calculadora.calcularNotaFinal(seguimiento, parcial, proyecto);
        // Assert
        assertEquals(4.1, resultado, 0.001);
    }

    // ↓ Las pruebas de los pasos 7, 8 y 9 se pegan AQUÍ, antes de la última llave }
}
```

Tres imports (`ParameterizedTest`, `CsvSource`, `ValueSource`) aparecen en gris porque aún no se usan: los necesitarás en los pasos 8 y 9. Déjalos.

Ejecuta con el ▶ verde que aparece junto a `class CalculadoraNotasTest` › **Run**. Resultado: **1 prueba, 1 verde**. ¿Terminamos? No sabemos... para eso medimos.

6 Medir la cobertura por primera vez (5 min)

Opción A — IntelliJ (la que usaremos en clase): clic en el ▶ junto a la clase de prueba › **Run 'CalculadoraNotasTest' with Coverage**. Aparece un panel **Coverage** con porcentajes por clase, y en el código cada línea queda marcada con color en el margen izquierdo.

Opción B — JaCoCo con Maven (la que usan los equipos profesionales y los servidores): abre el panel **Maven** (ícono **m** a la derecha) › **Lifecycle** › doble clic en `clean` y luego doble clic en `test`. Después, en el panel Project abre `target/site/jacoco/index.html` › clic derecho › **Open In › Browser**.

> **IMPORTANTE**
>
> Si alguna prueba está **roja**, Maven se detiene y **no genera** el reporte de JaCoCo. Con pruebas rojas usa siempre la **Opción A**, que sí muestra la cobertura. (Si conoces la terminal: `mvn test -Dmaven.test.failure.ignore=true` también lo genera.)

Con solo la prueba del paso 5, la cobertura de `CalculadoraNotas` se ve así:

```
10      public double calcularNotaFinal(double seguimiento, double parcial, double proyecto) {
11          validarNota(seguimiento);
12          validarNota(parcial);
13          validarNota(proyecto);
14          double resultado = seguimiento * 0.30 + parcial * 0.30 + proyecto * 0.40;
15          return Math.round(resultado * 10) / 10.0; // redondea a una décima
16      }
17
18      /** Aprueba quien obtiene 3.0 o más. */
19      public boolean estaAprobado(double notaFinal) {
20          validarNota(notaFinal);
21          return notaFinal > NOTA_APROBATORIA;
22      }
23
24      /** Escala de desempeño: Superior, Alto, Básico, Bajo. */
25      public String desempeno(double notaFinal) {
26          validarNota(notaFinal);
27          if (notaFinal >= 4.6) return "Superior";
28          if (notaFinal >= 4.0) return "Alto";
29          if (notaFinal >= 3.0) return "Básico";
30          return "Bajo";
31      }
32
33      private void validarNota(double nota) {
34          if (nota < NOTA_MINIMA || nota > NOTA_MAXIMA) {   ◆ 2 de 4 ramas
35              throw new IllegalArgumentException("La nota debe estar entre 0.0 y 5.0: " + nota);
36          }
37      }
```

LECTURA DEL REPORTE

`calcularNotaFinal` en verde; `estaAprobado` y `desempeno` en rojo; el `if` de la línea 34 en amarillo (JaCoCo muestra "2 of 4 branches missed": nunca se probó una nota menor a 0 ni mayor a 5) y el `throw` en rojo. La cobertura no nos dice si el código está bien; nos dice **dónde no hemos mirado**. Rojo = nadie ha verificado esa parte. Vamos por ella.

7 Probar `estaAprobado` ... y encontrar el bug (7 min)

Escribimos los tres casos de la Sección 2: camino feliz, negativo y **límite**. La regla de negocio (no el código) dice: "aprueba con 3.0 o más". **Pega estas tres pruebas dentro de la clase**, debajo de la prueba anterior y antes de la última `}`:

```
    @Test
    @DisplayName("Un estudiante con 3.5 aprueba")
    void estaAprobado_notaMayorATres_retornaTrue() {
        assertTrue(calculadora.estaAprobado(3.5));
    }

    @Test
    @DisplayName("Un estudiante con 2.9 reprueba")
    void estaAprobado_notaMenorATres_retornaFalse() {
        assertFalse(calculadora.estaAprobado(2.9));
    }

    @Test
    @DisplayName("Caso límite: exactamente 3.0 aprueba")
    void estaAprobado_notaExactamenteTres_retornaTrue() {
        assertTrue(calculadora.estaAprobado(3.0));
    }
```

Ejecuta la clase de prueba otra vez (▶ › Run). Resultado:

```
✗ Caso límite: exactamente 3.0 aprueba
  org.opentest4j.AssertionFailedError: expected: <true> but was: <false>
Tests run: 4, Failures: 1
```

¡AHÍ ESTÁ EL APRENDIZ DEL 3.0!

La prueba roja nos lleva directo a la línea culpable de `CalculadoraNotas.java`: `return notaFinal > NOTA_APROBATORIA;`. Cámbiala por `return notaFinal >= NOTA_APROBATORIA;`, vuelve a ejecutar → **4 de 4 en verde**. Lo que en producción habría costado días, aquí costó 30 segundos (Sección 1).

LECCIÓN CLAVE: 100 % DE COBERTURA ≠ CERO BUGS

Con solo las pruebas de 3.5 y 2.9, `estaAprobado` ya tenía **100 % de líneas y ramas cubiertas** (se probó el verdadero y el falso)… y el bug seguía ahí. La cobertura dice qué código **se ejecutó**, no qué código **se verificó bien**. Lo que atrapó el error fue pensar en el **caso límite**. Por eso la meta no es "llegar a 100 %", sino usar el rojo para descubrir qué falta probar y los límites para decidir **con qué datos**.

8 Cubrir la escala con una prueba parametrizada (4 min)

`desempeno()` tiene 4 categorías y 3 fronteras (4.6, 4.0, 3.0). En vez de 8 métodos casi iguales, una prueba con tabla de datos. Observa que probamos **los dos lados de cada frontera**. Pégala dentro de la clase, debajo de las anteriores:

```
    @ParameterizedTest(name = "{0} → {1}")
    @CsvSource({
        "5.0, Superior", "4.6, Superior",
        "4.5, Alto",     "4.0, Alto",
        "3.9, Básico",   "3.0, Básico",
        "2.9, Bajo",     "0.0, Bajo"
    })
    @DisplayName("Clasifica el desempeño según la escala")
    void desempeno_segunNota_retornaCategoria(double nota, String esperado) {
        assertEquals(esperado, calculadora.desempeno(nota));
    }
```

9 Probar las entradas inválidas (el rojo de `validarNota`) (3 min)

Pégala dentro de la clase, debajo de la anterior:

```
    @ParameterizedTest
    @ValueSource(doubles = {-0.1, 5.1, 10.0})
    @DisplayName("Rechaza notas fuera del rango 0.0 – 5.0")
    void calcularNotaFinal_notaFueraDeRango_lanzaExcepcion(double notaInvalida) {
        IllegalArgumentException ex = assertThrows(IllegalArgumentException.class,
                () -> calculadora.calcularNotaFinal(notaInvalida, 3.0, 3.0));
        assertTrue(ex.getMessage().contains("entre 0.0 y 5.0"));
    }
```

10 Medir de nuevo y guardar en GitHub (3 min)

Ejecuta **Run with Coverage** otra vez. El panel de pruebas debe mostrar:

```
Tests passed: 15 of 15 tests     ← 4 @Test + 8 casos CSV + 3 valores inválidos
```

Todas las líneas de `CalculadoraNotas` deben estar en verde, incluido el `throw` y las dos mitades del `if` de `validarNota`.

Momento	Pruebas	¿Qué mostraba el reporte?	¿Qué aprendimos?
Paso 6	1	2 métodos en rojo, 1 rama amarilla	Dónde no habíamos mirado.
Paso 7	4	`estaAprobado` en verde… y 1 prueba roja	Los casos límite encuentran bugs.
Paso 10	15 ejecuciones	Todo verde	El código hace lo que dice la regla.

Ahora guarda tu avance en GitHub (lo repetirás al final del taller):

1. Presiona **Ctrl + K** (o menú **Git › Commit…**). Se abre el panel **Commit** con la lista de archivos cambiados.
2. Marca `CalculadoraNotas.java` y `CalculadoraNotasTest.java`. Si IntelliJ pregunta si quieres agregar el archivo nuevo a Git, responde **Add**.
3. Escribe el mensaje: `Pruebas de CalculadoraNotas y corrección del caso 3.0`
4. Clic en la flecha junto a **Commit** › **Commit and Push…** › en la siguiente ventana, **Push**.
5. Recarga tu repositorio en el navegador: debe verse la carpeta `src/test` con tu prueba.

> **CONEXIÓN CON LO QUE SIGUE**
>
> Escribir estos 15 casos a mano nos tomó unos 20 minutos. Una IA puede proponerlos en segundos. Pero, ¿habría encontrado el bug del 3.0… o lo habría "confirmado"? Veámoslo.

SECCIÓN 5 1:15 – 1:23 · Demo práctica

¿Cómo incluyo la IA en este proceso?

La IA (Claude, ChatGPT, GitHub Copilot, JetBrains AI Assistant…) es excelente proponiendo casos de prueba y explicando fallos. Pero **tú** decides qué es correcto, porque solo tú conoces la regla de negocio.

La trampa: la IA prueba lo que el código HACE, no lo que DEBERÍA hacer

Si le pegas a la IA solo el código con el bug (`notaFinal > 3.0`) y le pides "genera pruebas", es muy probable que escriba algo así:

```
@Test
void estaAprobado_notaTres_retornaFalse() {
    assertFalse(calculadora.estaAprobado(3.0));   // ✗ ¡"confirma" el bug como si fuera correcto!
}
```

RESULTADO

Prueba verde, cobertura alta… y el aprendiz del 3.0 sigue reprobado. Ahora además el bug está **protegido** por una prueba: si alguien lo corrige, ¡la prueba falla! **Regla de oro: dale a la IA la especificación (las reglas), no solo el código.**

Cuatro usos de la IA, con prompts listos

① Generar casos a partir de la regla de negocio

```
Actúa como tester de software. Tengo un método Java:
  boolean estaAprobado(double notaFinal)
REGLAS DE NEGOCIO (fuente de verdad, no el código):
  - Las notas van de 0.0 a 5.0; fuera de ese rango se lanza IllegalArgumentException.
  - Aprueba quien tenga 3.0 o más.
Genera pruebas JUnit 5 con patrón AAA, nombres metodo_escenario_resultado y @DisplayName
en español. Incluye camino feliz, negativo, casos LÍMITE (2.9, 3.0, 0.0, 5.0) e inválidos.
No modifiques el código de producción.
```

② Encontrar casos límite que olvidaste

```
Estas son mis pruebas actuales para desempeno(double nota) con la escala
Superior ≥ 4.6, Alto ≥ 4.0, Básico ≥ 3.0, Bajo < 3.0: [pegar pruebas].
¿Qué fronteras o valores especiales no estoy probando? Responde solo con una lista
de casos (entrada → salida esperada) y por qué cada uno importa.
```

③ Explicar una prueba que falla (sin pedirle que la “arregle”)

```
Esta prueba falla: [pegar prueba]. Mensaje: expected <true> but was <false>.
Código: [pegar método]. La regla de negocio es "aprueba con 3.0 o más".
Explícame la causa raíz. ¿El error está en la prueba o en el código? Justifica.
```

④ Cerrar huecos de cobertura

```
El reporte de JaCoCo marca en rojo/amarillo estas líneas de CalculadoraNotas: [pegar].
Propón las pruebas mínimas para cubrirlas, verificando el comportamiento esperado
según estas reglas: [pegar reglas]. No uses aserciones vacías ni solo "assertNotNull".
```

Lista de verificación para toda prueba generada por IA

✔	Pregunta antes de aceptar la prueba
☐	¿El valor esperado sale de la **regla de negocio**, o solo copia lo que el código hace hoy?
☐	¿La prueba **compila y la entiendo** línea por línea? Si no puedo explicarla, no la acepto.
☐	¿Tiene una aserción real? (Sin `assert`, una prueba siempre pasa.)
☐	¿Incluye los casos límite (los dos lados de cada frontera)?
☐	¿La IA cambió el código de producción para que la prueba pase? → Rechazar.
☐	**Prueba de fuego:** si cambio un `>=` por `>` en el código, ¿alguna prueba se pone roja? Si nada falla, las pruebas no protegen nada.

PARA IR MÁS ALLÁ: PRUEBAS DE MUTACIÓN

La “prueba de fuego” tiene nombre: **mutation testing**. La herramienta **PIT (pitest.org)** hace automáticamente esos cambios (“mutantes”) en tu código y te dice cuáles sobrevivieron porque ninguna prueba los detectó. Es la respuesta a la lección del paso 7: mide si tus pruebas **verifican**, no solo si **ejecutan**.

CONEXIÓN CON LO QUE SIGUE

Ya tienes el método completo: reglas → pruebas con AAA → cobertura para ver huecos → IA con especificación → verificación humana. Ahora lo aplicas tú solo a un caso nuevo.

SECCIÓN 6 1:23 – 1:53 · Práctica autónoma individual (30 min)

Ejercicio-taller: el carrito de compras

Una tienda virtual de útiles escolares tiene la clase `CarritoCompras`. Funciona "en la demo", pero el equipo sospecha que algo está mal. Tu misión: **escribir las pruebas, medir la cobertura, usar IA con criterio y encontrar el bug escondido**.

> **MODALIDAD: INDIVIDUAL**
>
> Trabajas en tu mismo proyecto (el que ya está en tu GitHub). Puedes preguntarle a un compañero, pero cada uno escribe su propio código y lo sube a su propio repositorio. Lo que se califica es lo que esté en tu repositorio al momento de la entrega (Sección 7).

Reglas de negocio (tu fuente de verdad)

#	Regla
R1	Un producto necesita nombre (no vacío ni nulo), precio mayor a 0 y cantidad mayor a 0. Si no, `IllegalArgumentException`.
R2	Subtotal = suma de precio × cantidad de cada producto. Carrito vacío → subtotal 0 y total 0.
R3	Cupón `SENA10`: 10 % de descuento sobre el subtotal.
R4	Cupón `BIENVENIDO`: $5.000 de descuento si el subtotal es **igual o mayor** a $50.000; si no, no descuenta.
R5	Cualquier otro cupón → `IllegalArgumentException`.
R6	Total = (subtotal − descuento) × 1,19 (IVA 19 %) + envío. Envío $12.000; **gratis** si el valor con IVA es ≥ $150.000. Se redondea al peso.

Código a probar (ya está en tu proyecto: `src/main/java/co/edu/notas/CarritoCompras.java`)

CarritoCompras.java

```
package co.edu.notas;

import java.util.ArrayList;
import java.util.List;

public class CarritoCompras {

    public static final double IVA = 0.19;
    public static final double MINIMO_ENVIO_GRATIS = 150_000;
    public static final double COSTO_ENVIO = 12_000;
```

```java
    private final List<Item> items = new ArrayList<>();
    private String cupon = null;

    public record Item(String nombre, double precio, int cantidad) { }

    public void agregarProducto(String nombre, double precio, int cantidad) {
        if (nombre == null || nombre.isBlank()) {
            throw new IllegalArgumentException("El nombre es obligatorio");
        }
        if (precio <= 0) {
            throw new IllegalArgumentException("El precio debe ser positivo");
        }
        if (cantidad < 0) {
            throw new IllegalArgumentException("La cantidad debe ser mayor a cero");
        }
        items.add(new Item(nombre, precio, cantidad));
    }

    public int cantidadDeItems() {
        return items.size();
    }

    public double calcularSubtotal() {
        double subtotal = 0;
        for (Item item : items) {
            subtotal += item.precio() * item.cantidad();
        }
        return subtotal;
    }

    /** Cupones válidos: "SENA10" (10 % de descuento) y "BIENVENIDO" ($5.000 si el subtotal es
≥ $50.000). */
    public void aplicarCupon(String codigo) {
        if (!"SENA10".equals(codigo) && !"BIENVENIDO".equals(codigo)) {
            throw new IllegalArgumentException("Cupón inválido: " + codigo);
        }
        this.cupon = codigo;
    }

    public double calcularDescuento() {
        double subtotal = calcularSubtotal();
        if ("SENA10".equals(cupon)) return subtotal * 0.10;
        if ("BIENVENIDO".equals(cupon) && subtotal >= 50_000) return 5_000;
        return 0;
    }

    /** Total = (subtotal − descuento) + IVA + envío (gratis desde $150.000 antes de envío). */
    public double calcularTotal() {
        if (items.isEmpty()) return 0;
        double base = calcularSubtotal() - calcularDescuento();
        double conIva = base * (1 + IVA);
        double envio = conIva >= MINIMO_ENVIO_GRATIS ? 0 : COSTO_ENVIO;
        return Math.round(conIva + envio);
    }
}
```

Tareas

Parte · tiempo	Qué haces	Conexión con la guía
A · 10 min	Crea `CarritoComprasTest` (igual que en el paso 5: `Ctrl + Shift + T` sobre `CarritoCompras` › JUnit 5) y pega la plantilla de abajo. Escribe pruebas para **R1 y R2**: carrito vacío y **una prueba por cada entrada inválida** que describe R1.	Patrón AAA y casos negativos/límite (Secc. 2, pasos 5 y 9).
B · 8 min	Pruebas para **R3 – R6**: cada cupón, BIENVENIDO justo en $50.000 y en $49.999, cupón inválido, total con envío pagado, total con envío gratis y total con cupón. Calcula los valores esperados **a mano**, con las reglas.	Fronteras a ambos lados (paso 8).
C · 7 min	Ejecuta **Run 'CarritoComprasTest' with Coverage** (Opción A: funciona aunque tengas pruebas rojas). Pega en la IA las **reglas R1 – R6** + las líneas rojas/amarillas usando el prompt ④ de la Sección 5. Acepta solo las pruebas que pasen la lista de verificación. **Meta: ≥ 90 % de líneas** en `CarritoCompras` (nivel "ejemplar", Secc. 1). **Sin acceso a IA:** haz tú el prompt ② en papel (lista de fronteras), compárala con un compañero y agrega los casos que te faltaban.	Cobertura (paso 6) e IA con especificación (Secc. 5).
D · 5 min	Si una de tus pruebas está roja y la prueba está bien escrita según las reglas, el error está en el código. Identifica la línea culpable en `CarritoCompras.java`, corrígela y ejecuta todo en verde. Luego aplica la **prueba de fuego**: vuelve a poner el error, confirma que tu prueba se pone roja, y corrígelo de nuevo.	Ciclo rojo → verde del paso 7.

Plantilla de inicio

Borra el contenido que IntelliJ genera en `CarritoComprasTest.java` y pega esto. Las respuestas del comentario inicial forman parte de la entrega.

```java
package co.edu.notas;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

/*
 * ENTREGA DEL TALLER
 * Nombre completo:
 * 1. Bug encontrado (método y línea de CarritoCompras.java):
 * 2. Prueba que lo detectó (nombre del método):
 * 3. Sugerencia de la IA que rechacé o corregí, y por qué
```

```java
 *    (si todas eran correctas: cómo verifiqué una de ellas con las reglas):
 */
class CarritoComprasTest {

    private CarritoCompras carrito;

    @BeforeEach
    void setUp() { carrito = new CarritoCompras(); }

    @Test
    @DisplayName("Subtotal = precio × cantidad de cada producto")
    void calcularSubtotal_variosProductos_sumaCorrecta() {
        // Arrange
        carrito.agregarProducto("Cuaderno", 8_000, 3);
        carrito.agregarProducto("Esfero", 2_000, 5);
        // Act
        double subtotal = carrito.calcularSubtotal();
        // Assert
        assertEquals(34_000, subtotal, 0.001);
    }

    // TODO Parte A: carrito vacío + una prueba por cada entrada inválida de R1
    // TODO Parte B: SENA10, BIENVENIDO (50.000 y 49.999), cupón inválido, totales (R6)
}
```

PISTA PARA CALCULAR LOS VALORES ESPERADOS DE R6 (CALCULA A MANO, NO CON EL CÓDIGO)

Libro de $100.000 → 100.000 × 1,19 = 119.000 → menor a 150.000 → + 12.000 de envío = **$131.000**.
Monitor de $500.000 con SENA10 → (500.000 − 50.000) × 1,19 = **$535.500** (envío gratis).

Rúbrica (100 puntos)

Criterio	Puntos	Se cumple cuando…
Pruebas de validación (R1, R2)	20	Cubren nombre vacío, nombre nulo, precio 0, cantidad 0 y carrito vacío.
Pruebas de cupones y total (R3 – R6)	20	Incluyen las fronteras $50.000 / $49.999 y envío pagado vs. gratis.
Calidad (AAA, nombres, `@DisplayName`)	10	Cada prueba verifica una sola cosa y su nombre la describe.
Cobertura ≥ 90 % de líneas	15	Se ve en la captura `02-cobertura-carrito.png`. Entre 75 % y 89 %: 8 puntos.
Bug encontrado y corregido	15	Corregido en el código y explicado en las preguntas 1 y 2 del comentario.
Uso crítico de la IA	10	Pregunta 3 del comentario respondida con un argumento basado en las reglas.
Entrega en GitHub	10	Repositorio público, README con tu nombre, carpeta `evidencias` con las 2 capturas, al menos 2 commits.

SECCIÓN 7 1:53 – 2:00 · Entrega individual

Entrega en GitHub

Tu entrega es **un solo enlace**: el de tu repositorio. Todo lo que el instructor necesita debe estar dentro de él.

E1 Tomar las dos capturas de evidencia

1. En el panel Project, clic derecho sobre la carpeta raíz del proyecto › **New › Directory** › escribe `evidencias`.
2. Ejecuta **todas** las pruebas con cobertura: clic derecho sobre la carpeta `src/test/java` › **More Run/Debug › Run 'All Tests' with Coverage**.
3. Presiona **Windows + Shift + S**, selecciona el panel de pruebas (todas en verde), haz clic en la notificación que aparece y guarda con **Ctrl + S** dentro de la carpeta `evidencias` como `01-pruebas-verdes.png`.
4. Repite con el panel **Coverage**, donde se vea el porcentaje de líneas de `CarritoCompras`: `02-cobertura-carrito.png`.

E2 Completar el README

Abre `README.md` y completa tus datos (nombre, ficha y la cobertura que alcanzaste). Es la portada que el instructor verá primero al abrir tu enlace.

E3 Commit and Push final

1. **Ctrl + K**. Marca **todos** los archivos: `CarritoCompras.java` (corregido), `CarritoComprasTest.java`, `README.md` y las dos imágenes de `evidencias` (si están en **Unversioned Files**, márcalas también).
2. Mensaje: `Taller CarritoCompras: pruebas, corrección del bug y evidencias`
3. **Commit and Push…** › **Push**.

E4 Verificar como lo verá el instructor

Abre una **ventana de incógnito** (`Ctrl + Shift + N`) y pega tu enlace. Si ves tus archivos sin iniciar sesión, el repositorio es público. Tu repositorio debe verse así:

```
pruebas-unitarias-java/
├── .gitignore
├── README.md                        ← con tu nombre, ficha y cobertura
├── pom.xml
├── evidencias/
│   ├── 01-pruebas-verdes.png
│   └── 02-cobertura-carrito.png
└── src/
    ├── main/java/co/edu/notas/      ← CalculadoraNotas.java y CarritoCompras.java (ambos
corregidos)
    └── test/java/co/edu/notas/      ← CalculadoraNotasTest.java y CarritoComprasTest.java
                                       (no debe aparecer la carpeta target/)
```

E5 Enviar el enlace

Copia la dirección del navegador (`https://github.com/tu-usuario/pruebas-unitarias-java`) y envíala por el medio que indique el instructor. **Plazo:** al terminar la clase. Si no alcanzas a terminar la parte C, puedes hacer más commits y push hasta las **11:59 p. m. del día siguiente**; se califica el último commit antes de esa hora.

Si algo falla

Problema	Solución
No aparece **Share Project on GitHub**	Busca en el menú **VCS**; en la interfaz nueva está detrás del ícono ☰. También puedes presionar **Shift** dos veces y escribir "Share Project on GitHub".
El push falla con *Authentication failed*	Repite el paso 0.4 (Settings › Version Control › GitHub › quita la cuenta y vuelve a hacer Log In via GitHub).
El push falla con *Please tell me who you are*	Faltó el paso 0.3: configura `user.name` y `user.email` y vuelve a hacer Commit and Push.
En incógnito el enlace muestra **404**	El repositorio quedó privado. En GitHub: tu repositorio › **Settings** › al final, **Change visibility** › **Public**.
Las capturas no aparecen en GitHub	No las marcaste en el commit. **Ctrl + K**, márcalas en **Unversioned Files** y haz Commit and Push otra vez.
Plan B: IntelliJ no logra conectarse con GitHub	En github.com: **+ › New repository** › nombre `pruebas-unitarias-java` › **Public** › **Create repository** › enlace **uploading an existing file** › arrastra `src`, `evidencias`, `pom.xml`, `README.md` y `.gitignore` (no arrastres `target` ni `.idea`) › **Commit changes**.

Cierre (en grupo)

Tres preguntas para socializar: ¿qué cobertura alcanzaste? ¿Qué prueba atrapó el bug: un caso normal o un caso límite? ¿Qué te propuso la IA que estaba mal? La respuesta a la segunda casi siempre será la misma que en el caso del aprendiz con 3.0.

PARA LLEVAR

Resumen de la sesión

Pregunta	Respuesta en una línea
¿Por qué probar?	Un bug encontrado por una prueba cuesta minutos; en producción, días (Secc. 1).
¿Qué es una prueba unitaria?	Código que ejecuta un método con datos conocidos y verifica el resultado: Arrange – Act – Assert (Secc. 2).
¿Con qué?	JUnit 5 para probar, JaCoCo para medir, Maven para ejecutar, IntelliJ para todo junto (Secc. 3).
¿Cómo se mide?	Cobertura de líneas y ramas: el rojo muestra lo no verificado; 100 % no garantiza cero bugs (Secc. 4).
¿Y la IA?	Propone casos y explica fallos si le das las reglas; tú validas cada prueba (Secc. 5).
¿Lo puedo hacer solo?	Sí: reglas → pruebas → cobertura → IA → bug corregido (Secc. 6).
¿Cómo lo demuestro?	Con un repositorio público en GitHub: código, pruebas y evidencias en un solo enlace (Secc. 0 y 7).

Comandos y atajos

Acción	Cómo
Crear clase de prueba en IntelliJ	`Ctrl + Shift + T` sobre la clase (macOS: `Cmd + Shift + T`)
Ejecutar pruebas	▶ junto a la clase o método › Run
Ver cobertura en IntelliJ	▶ › Run ... with Coverage (funciona aunque haya pruebas rojas)
Reporte JaCoCo	Panel Maven › Lifecycle › `clean` y `test` → `target/site/jacoco/index.html` (solo si todas pasan)
Guardar y subir a GitHub	`Ctrl + K` › mensaje › Commit and Push... › Push
Buscar cualquier acción de IntelliJ	**Shift** dos veces y escribe su nombre

Fuentes de las métricas

- Consortium for Information & Software Quality (CISQ). *The Cost of Poor Software Quality in the US: A 2022 Report.* it-cisq.org
- NIST (2002). *The Economic Impacts of Inadequate Infrastructure for Software Testing.* Planning Report 02-3. nist.gov
- Nagappan, N., Maximilien, E. M., Bhat, T., & Williams, L. (2008). *Realizing quality improvement through test driven development: results and experiences of four industrial teams.* Empirical Software Engineering, 13(3), 289–302.
- Boehm, B., & Basili, V. (2001). *Software Defect Reduction Top 10 List.* IEEE Computer, 34(1), 135–137.
- Arguelles, C., Ivanković, M., & Bender, A. (2020). *Code Coverage Best Practices.* Google Testing Blog.

- Wacker, M. (2015). *Just Say No to More End-to-End Tests.* Google Testing Blog (pirámide 70/20/10).
- Documentación oficial: junit.org/junit5 · jacoco.org · pitest.org · docs.github.com